Extended Abstract

Process Rewards and Tool Use: Two Extensions to RLOO Fine-Tuning for Math Reasoning

Du Li Michael Yang

Motivation. The default project trains Qwen2.5–0.5B on Countdown, a math puzzle that asks the model to combine a small set of integers into a target value. The default recipe – SFT warm-start followed by RLOO with a rule-based outcome verifier – gives a single bit of feedback per rollout: the equation either hits the target or it does not. This sparseness raises two questions: (1) Can a denser *step-level* reward speed learning when correct outcomes are rare? (2) Can a *calculator tool* help with arithmetic the model already handles? We test both at the 0.5B scale, where the bottleneck is small enough to expose what each mechanism contributes.

Method. We build two extensions on top of RLOO (leave-one-out baseline, importance-weight clip, KL penalty, entropy bonus). *Extension 1 (PRM, primary):* we parse the `<think>` trace into arithmetic steps, score each against a running number pool, and add a bounded partial-credit bonus to the outcome reward. The pool grows with each result so wrong steps do not cascade-penalize later lines. *Extension 2 (TIR, secondary):* the model emits `<calc>EXPR</calc>`, we evaluate `EXPR` via sandboxed Python, and inject `<out>VAL</out>` back (loss-masked). We evaluate TIR via a $2\times2\times2$ matrix crossing trained-tool $\times$ eval-tool $\times$ SFT/RLOO.

Implementation. PRM is a drop-in scorer (single-line import swap): regex step parser plus number-pool tracer. TIR adds an interpreter-in-the-loop sampler (`</calc>` stop string), character-offset loss mask on `<out>` spans, and a corpus rewriter producing 11,839 `<calc>/<out>` rewrites across 1,000 SFT traces. All RLOO runs: K=8, batch 128, lr 1×10^{-5}, KL/entropy coefficients 10^{-3}, 100 steps.

Results. *PRM (primary; full).* **PRM-Exp** (expectile-mode, α=0.5) achieves pass@1 $= 0.676$ at step 70, a $+6.3$pp gain over the RLOO-ORM outcome-only baseline (0.613) and $+2.5$pp over extended outcome-only training at matched compute (RLOO-ORM^{+}, 0.651). A six-variant ablation spanning clipped, continuous, mixture, and expectile aggregation modes reveals that *reward scale* (α) dominates aggregation design: the original average-mode PRM performs similarly (0.670), while the expectile model at over-scaled α=2.0 collapses to 0.643. *TIR (secondary; full).* The $2\times2\times2$ matrix on Countdown 3-to-4 (50 prompts $\times$ 16 samples per cell) gives a sharp result: **text-only RLOO beats RLOO-TIR** head-to-head (0.520 vs. 0.413), even though SFT-TIR invokes the calculator on 99.9% of rollouts with zero errors. Disabling the calculator at inference drops RLOO-TIR from 0.413 to 0.236; the model learned the tool, not the task. The best cell is text-RLOO with the calculator turned on only at inference (0.609).

Discussion. *PRM*: Reward scale (α) dominates aggregation design. The step parser's coarse three-symbol alphabet ($\{+1.0, -1.0, -1.5\}$) creates an information bottleneck: all modes plateau near the same pass@1 ceiling. Surpassing this likely requires a learned verifier. *TIR*: The training-time cost has two causes: (1) a 0.5B model already handles simple arithmetic, so the calculator adds $\sim$18–21 tokens without fixing real errors (*bottleneck mismatch*); (2) `<calc>/<out>` blocks narrow the on-policy stream and a vLLM$\leftrightarrow$HuggingFace log-prob mismatch biases the importance ratio to $\sim$0.45 (*distribution shift*). PRM avoids both by leaving rollouts unchanged.

Conclusion. Two extensions, two verdicts. Process rewards are a *structural* fix for sparse outcomes: they cost no inference-time tokens and leave the rollout shape unchanged. Tools are a *situational* fix: on Countdown the bottleneck is search, not arithmetic, so a calculator trained into the policy pays its overhead for a capability the base policy already had. Future work: joint PRM–policy learning, and re-running the matrix where arithmetic actually is the binding constraint.

Process Rewards and Tool Use: Two Extensions to RLOO Fine-Tuning for Math Reasoning

Du Li
Department of Computer Science
Stanford University
li2723@stanford.edu

Michael Yang
Department of Computer Science
Stanford University
myang243@stanford.edu

Abstract

The default CS224R project trains Qwen2.5–0.5B on Countdown using RLOO with a rule-based outcome verifier. Outcome reward is sparse, and the model sometimes makes basic arithmetic slips. We build two extensions that target each weakness directly. **Extension 1 (PRM, primary)** adds a bounded, average-mode process reward that grades each parsed arithmetic step against a running pool of numbers the model is allowed to use, and adds the result to the outcome reward without changing rollout content. **Extension 2 (TIR, secondary)** gives the model a `<calc>EXPR</calc>` calculator backed by a sandboxed Python `eval`, injects `<out>VAL</out>` back into the rollout, and masks injected tokens from the loss. We evaluate TIR with a $2\times2\times2$ ablation matrix that separates training-time from inference-time tool exposure, and find a sharp result: tool training costs 11 pass@1 points (text-RLOO 0.520 vs. RLOO-TIR 0.413) while the best overall cell is text-RLOO with the calculator turned on *only at inference* (0.609). The PRM ablation covers six formulations (PRM-Avg, PRM-Clip, PRM-Soft, PRM-Mix, PRM-Exp at two α scales); the best, **PRM-Exp** at pass@1 $= 0.676$, leads extended outcome-only training at matched compute by $+2.5$pp, and a scale ablation on the expectile variant reveals that bonus amplitude α dominates aggregation design.

1 Introduction

Reinforcement learning from a rule-based verifier has become a standard final stage for math-reasoning language models, from DeepSeek-Math Shao et al. [2024] to TinyZero Pan et al. [2025]. The default CS224R project distills this pipeline into a tractable setting: Qwen2.5–0.5B Qwen Team [2025] on Countdown, a small-integer arithmetic puzzle. The required RLOO Ahmadian et al. [2024] stage uses a single $\{0,\ 0.1,\ 1\}$-valued reward – zero for malformed output, 0.1 for the right format but wrong equation, 1 for an equation that uses each provided number exactly once and evaluates to the target.

This reward shape has two known weaknesses. (1) It is *outcome-only*, so a rollout that does five correct arithmetic steps and a sixth wrong one is graded the same as a rollout that fabricates numbers from the start. (2) It treats arithmetic as a hidden capability of the policy, so when the model slips on a multiplication or addition the only feedback is a zero at the end. The first weakness motivates a denser, step-level reward Lightman et al. [2023], Wang et al. [2024]. The second motivates giving the model a tool (i.e., the same calculator a person would use) and letting it delegate the arithmetic Schick et al. [2023], Gou et al. [2024], Gao et al. [2023].

We test both directions on top of the same RLOO implementation we built for the milestone. Two concrete research questions guide the work:

- **RQ1 (PRM).** Does a bounded, average-mode process reward that grades each parsed arithmetic step against a running number pool improve pass@1 over outcome-only RLOO at matched compute, without inducing length-hacking?
- **RQ2 (TIR).** Does training the policy with a calculator in the loop help on Countdown, or does the benefit (if any) come entirely from inference-time tool use on a text-only policy?

PRM is our *primary* extension; the policy keeps the same text-only rollout shape, so we can attribute any change in pass@1 to the reward signal alone. TIR is our *secondary* extension; both extensions are reported in full here (Sections 5.1–5.2).

The clearest contribution is the TIR ablation matrix (Section 5.2). Most TIR studies report a single "text vs. tool" delta; we factor the design along three axes (training-time tool exposure $\times$ inference-time tool access $\times$ SFT vs. RLOO). This separates the *tool protocol* capability from the *task* capability, and reveals that on Countdown the best cell is a text-only RLOO policy handed a calculator only at evaluation time.

2 Related Work

Outcome- vs. process-supervised RL. PRM800K Lightman et al. [2023] showed that step-level supervision outperforms outcome-only supervision on MATH-style problems; Math-Shepherd Wang et al. [2024] extended this to fully automatic step labels generated by Monte-Carlo roll-forwards from each prefix. Both line up step-level supervision with a ranker that then re-orders candidate solutions. Our PRM is structurally simpler: a rule-based parser that scores each line of the `<think>` trace, then a bounded bonus that is added to the outcome verifier the default project already uses. Rule-based step rewards are attractive at the 0.5B scale because they do not require an extra reward model and they leave the policy's rollout shape unchanged – so any contribution can be attributed to the gradient signal rather than to distribution shift, unlike TIR, which perturbs both.

Tool-integrated reasoning. Toolformer Schick et al. [2023] teaches a base model to insert API calls through self-supervised data construction. ToRA Gou et al. [2024] fine-tunes math LLMs to interleave natural-language reasoning with calls to a Python REPL and a symbolic solver, scaling from 7B to 70B; PAL Gao et al. [2023] reformulates chain-of-thought as Python programs so that arithmetic is offloaded to the interpreter. These works almost uniformly report tool use as a net positive, but they study harder math (GSM8K, MATH, TheoremQA) on much larger models. The closest setting to ours is the small-model arm of the ToRA ablations, which already shows a smaller gap. We argue, and the $2\times2\times2$ matrix supports, that the tool's contribution depends on whether the task's actual bottleneck is arithmetic at all.

RLOO and PPO-style RL for LLMs. RLOO Ahmadian et al. [2024] is the REINFORCE-style estimator we use, with a leave-one-out baseline over the K siblings sampled per prompt. It replaces both the critic of PPO Schulman et al. [2017] and the average-over-group baseline of GRPO Shao et al. [2024]. We pair it with a per-sequence importance weight (needed because rollouts come from vLLM Kwon et al. [2023] but the gradient is computed under the HuggingFace policy Qwen Team [2025]) and an analytical KL penalty against a frozen reference, following the default-project specification.

Preference vs. verifier-based RL. Our pipeline also follows the more general distinction between preference-based methods (DPO Rafailov et al. [2024], IPO Azar et al. [2023]) and verifier-based online RL (RLOO, GRPO). Both stages exist in the default project; the two extensions plug into the verifier-based stage, where the reward is a deterministic function the student implements.

3 Method

3.1 Default Recipe (RLOO)

Let π_θ be the policy and π_{ref} a frozen reference. For each prompt x we sample a group of K rollouts $y^{(1)}, \ldots, y^{(K)} \sim \pi_{\text{old}}(\cdot \mid x)$ from a behavior policy π_{old} – in our setup, the result of running vLLM on the current Hugging Face weights, which can produce slightly different token-level log-probs than

the trainer model (see Section 3.1). Let $R(y^{(i)}, x)$ be the reward for rollout i. The RLOO advantage is a leave-one-out estimate

$$A_i \;=\; R(y^{(i)}, x) \;-\; \frac{1}{K-1}\sum_{j\neq i} R(y^{(j)}, x), \tag{1}$$

and the surrogate loss is

$$\mathcal{L}_{\text{RLOO}}(\theta) \;=\; -\,\mathbb{E}[\,w_i \cdot A_i\,] \;-\; \beta\,\mathcal{H}(\pi_\theta) \;+\; \gamma\,\mathrm{KL}(\pi_\theta \| \pi_{\text{ref}}), \tag{2}$$

where $w_i = \pi_\theta(y^{(i)} \mid x)/\pi_{\text{old}}(y^{(i)} \mid x)$ is the per-sequence importance weight, $\mathcal{H}$ is the policy entropy, and β, γ are coefficients. Three implementation choices are worth calling out because both extensions inherit them.

Importance ratio. We compute $\log w_i$ as the difference of summed per-token log-probs and take $\exp(\min(\log w_i, \log 2))$ – a one-sided clip rather than the two-sided clip of PPO. The clip catches log-prob explosions from numerical drift between vLLM and HuggingFace without discarding the (more common) case where $w_i < 1$.

Analytical KL. We compute $\mathrm{KL}(\pi_\theta \| \pi_{\text{ref}})$ as the full-vocabulary analytical KL at each response token (via `torch.distributions.kl_divergence`) rather than as a Monte-Carlo estimator over sampled tokens. This is more accurate per step but doubles the forward pass; we set $\gamma = 10^{-3}$, which is small enough that the KL acts as a regularizer rather than a hard constraint.

Entropy bonus. We add $-\beta\,\mathcal{H}$ with $\beta = 10^{-3}$. With the one-sided ratio clip and a small KL coefficient, an explicit entropy term keeps the policy exploring across the 100 update steps. Both extensions use the same (β, γ).

3.2 Extension 1: Process Reward Model (Primary)

Why process rewards. The outcome reward is one bit: did the final equation hit the target? Early in training, most rollouts get zero and every gradient looks identical; the RLOO baseline is also zero, so there is no leave-one-out signal. PRM densifies the reward by grading the parts the model got right *inside* a rollout, without changing the rollout itself. Because the rollout shape is unchanged, the importance ratio and KL distribution under (2) also do not move – a structural advantage over TIR, which changes the rollout content.

Step parser. We parse the `<think>` trace line by line with a regex that matches `a op b = result`, where `op` $\in \{+, -, \times, \div\}$, with optional leading list markers ("1. ", "- ") and the `$` anchor rejecting trailing notes ("(too high)"). Lines that do not match are silently skipped. We keep a *number pool* that is seeded with the puzzle's input integers and grows with the absolute value of every *claimed* result, regardless of correctness. The pool-grows-with-claim rule prevents cascade penalization: a wrong step k does not double-penalize step $k{+}1$ for re-using the wrong value.

Per-step weights and aggregation. Each parsed step s_j receives a signed score $r(s_j)$ from Table 1. We aggregate the per-step scores into a single bonus and add it to the outcome reward:

$$r_{\text{PRM}}(\tau) \;=\; \alpha \cdot \mathrm{clip}\left(\frac{1}{m}\sum_{j=1}^{m} r(s_j),\; -1,\; +1\right), \qquad r_{\text{total}}(\tau) \;=\; r_{\text{ORM}}(\tau) \;+\; r_{\text{PRM}}(\tau), \tag{3}$$

with $\alpha = 0.4$ and m the number of parseable steps. If no steps parse, we fall back to the outcome reward only.

Dividing by m measures *quality density* rather than total quality, so the model cannot inflate the PRM bonus simply by generating longer `<think>` traces. The asymmetric weight on hallucinated numbers (-1.5 vs. -1.0 for arithmetic errors) reflects that fabricating operands is a more dangerous failure mode than slipping a multiplication – the verifier rejects a fabricated equation regardless of value, whereas an arithmetic slip can still produce a usable intermediate result.

Table 1: Per-step rewards for the average-mode PRM. Pool semantics: after each step, the absolute value of the *claimed* result is added to the pool regardless of correctness, which prevents cascade penalties on subsequent steps that reuse the wrong value.

Step classification	$r(s_j)$	Rationale
Valid operands, correct arithmetic	$+1.0$	Reward genuine progress
Valid operands, wrong arithmetic	-1.0	Penalize arithmetic errors
Hallucinated number (not in pool)	-1.5	Stronger penalty for fabrication
Unparseable line	0.0	Skip silently

Worked example. For the puzzle with input pool $\{44,\ 19,\ 35\}$ (no target shown):

```
44 + 19 = 63 +1.0 correct, operands in pool; pool now {44,19,35,63}
63 + 42 = 105 -1.5 42 not in pool; pool now {44,19,35,63,42,105}
35 - 19 = 17 -1.0 operands in pool, but actual answer is 16
```

With $m = 3$ steps and $\alpha = 0.4$, the unclipped average is $(1.0 - 1.5 - 1.0)/3 = -0.5$; clipping to $[-1, +1]$ and scaling by α gives $r_{\text{PRM}} = -0.20$, which is added to the binary outcome reward.

Variants explored. We systematically tested five aggregation families. Each replaces the PRM bonus calculation in Eq. 3 with a different formulation (with $r(s_j)$ from Table 1 unless otherwise noted):

- **PRM-Avg** (production): The average-mode baseline defined in Eq. 3, tested at α=0.4.
- **PRM-Clip**: Same as PRM-Avg but clips the raw per-step sum before averaging, rather than clipping the final average. It saturates on long correct traces, losing gradient information:

$$r_{\text{PRM-Clip}} \;=\; \alpha \cdot \text{clip}\left(\frac{1}{m}\text{clip}\Big(\sum_{j=1}^{m} r(s_j), -m, m\Big),\ -1,\ +1\right)$$

- **PRM-Soft**: Replaces the discrete -1.0 arithmetic-error score with a continuous penalty proportional to the relative error $e_{\text{rel}} = |\hat{y} - y|/\max(|y|, 1)$:

$$r_{\text{soft}}(s_j) = \begin{cases} +1.0 & \text{if correct} \\ -\min(e_{\text{rel}},\ 1.0) & \text{if arithmetic error} \\ -1.5 & \text{if hallucinated} \end{cases}$$

  On Countdown, arithmetic correctness is binary – a "nearly correct" intermediate value is strategically useless – so soft rewards falsely encourage such steps.

- **PRM-Mix**: A mixture of average-mode (quality density), veto (worst-step penalty), and accumulation (unweighted sum) components. The signals partially cancel and introduce noise:

$$r_{\text{PRM-Mix}} \;=\; \alpha \cdot \text{clip}\left(w_{\text{avg}}\frac{1}{m}\sum_{j=1}^{m} r(s_j) \;+\; w_{\text{min}}\min_j r(s_j) \;+\; w_{\text{sum}}\sum_{j=1}^{m} r(s_j),\ -1,\ +1\right)$$

  Tested at $\alpha \in \{0.5,\ 0.7\}$ with weights $(w_{\text{avg}}{=}0.5, w_{\text{min}}{=}0.25, w_{\text{sum}}{=}0.25)$.

- **PRM-Exp**: Asymmetric expectile weighting (τ=0.3), inspired by Implicit Q-Learning Kostrikov et al. [2022], encoding the observation that policies respond more strongly to penalties than rewards. Correct steps are down-weighted relative to errors:

$$w_\tau(x) = \begin{cases} \tau \cdot x & \text{if } x > 0 \\ (1-\tau)\cdot x & \text{if } x \le 0 \end{cases} \quad\Longrightarrow\quad r_{\text{PRM-Exp}} \;=\; \alpha\cdot\text{clip}\left(\frac{1}{m}\sum_{j=1}^{m} w_\tau(r(s_j)),\ -1,\ +1\right)$$

  Tested at $\alpha \in \{0.5,\ 2.0\}$; at α=2.0 the bonus exceeds ORM and collapses exploration (Table 2).

Integration and Codebase. PRM is a drop-in replacement for the verifier in `evaluation/countdown.py` – the reward signature is identical, so the only change to the RLOO trainer is a single import swap in `rloo_trainer/rloo.py`. This makes it easy to run PRM and outcome-only baselines from the same checkpoint with matched compute. Because the PRM variants (Clip, Soft, Mix, Exp) require structurally different aggregation logic and hardcoded hyperparameters, we maintain them on separate Git branches rather than unifying them into the single `main` branch configuration.

3.3 Extension 2: Tool-Integrated Reasoning (Secondary)

Tool protocol. The model emits `<calc>EXPR</calc>` anywhere in a rollout. We use `</calc>` as a vLLM stop string, evaluate `EXPR` through a sandboxed `calc(expr)` function – a regex character whitelist (`[0-9+-*/().\s]`), a 64-character length cap, and Python's `eval` with `__builtins__={}` and an empty locals dict – and inject `<out>VAL</out>` back into the rollout. If the expression fails the whitelist or raises, we inject `<out>err</out>`. Generation then resumes from the appended text. `</answer>` is the second stop string so the rollout ends cleanly once the model emits its final equation.

Interpreter-in-the-loop sampler. The sampler does a single round of native $n{=}K$ generation, then loops: for every rollout that stopped on `</calc>`, parse the last `<calc>...</calc>`, call `calc`, append `<out>val</out>`, and re-queue the rollout for a continuation pass ($n{=}1$, same temperature, smaller per-rollout token budget). We cap the loop at `max_rounds=8` rounds per rollout. Importantly, the returned sequence log-prob sums *only* over tokens the model actually emitted – injected `<out>...</out>` tokens are excluded – so the importance ratio in Eq. 2 compares apples to apples between sampler and trainer.

`<out>` **loss masking.** At training time we additionally apply a character-offset mask that zeros the loss on every token inside an `<out>...</out>` span (opening tag, payload, closing tag). The mask is produced by running the SFT tokenizer with `return_offsets_mapping=True`, walking each token's character span, and zeroing the mask if that span falls inside any `<out>...</out>` match. We mask the entire span, tags included, because the tags are deterministically injected at inference time and the model never emits them; training on them would teach the model to predict tokens it does not actually produce.

SFT corpus augmentation. For SFT-TIR we rewrite the `cog_behav_all_strategies` corpus with a regex that matches `a op b = result`; for every span where `calc(a op b)` agrees with the trace's claimed result, we rewrite the right-hand side as `<calc>a op b</calc><out>result</out>`. Spans where the trace itself was numerically wrong are left untouched – we do not want the model to internalize "delegate only when you were already right." Over the 1,000 train traces this produces 11,839 rewrites (11.84 per trace on average), with 620 spans skipped because the trace and `calc` disagreed. The output is a JSONL pair the SFT loader reads via `load_dataset("json", data_files=...)`.

The $2{\times}2{\times}2$ matrix. Most TIR studies compare a single "text vs. tool" pair, but two effects get conflated: the policy's *tool-protocol skill* (knowing when and how to call the calculator) and its *task skill* (knowing how to search the Countdown space at all). We factor the design along three binary axes:

1. **Trained tool** $\in \{$no, yes$\}$: whether the calculator was present during fine-tuning (text-SFT/RLOO vs. SFT-TIR/RLOO-TIR).
2. **Eval tool** $\in \{$no, yes$\}$: whether the calculator is available at inference time (vanilla sampler vs. interpreter sampler).
3. **Stage** $\in \{$SFT, RLOO$\}$: whether the policy has been RLOO-fine-tuned on Countdown after the SFT warm-start.

The eight cells fully separate the two skills: e.g. "trained=no, eval=yes" tests whether a tool-naive model can use a calculator zero-shot; "trained=yes, eval=no" tests whether tool training imparted a residual task ability that survives the tool being taken away.

Table 2: PRM ablation: best pass@1 within 80 training steps. Δ is the absolute gain vs. RLOO-ORM (0.613). † marks the variant that fails to exceed the no-PRM baseline.

Method	Aggregation	α	Best pass@1	Δ
RLOO-ORM	Outcome only	—	0.613	—
RLOO-ORM$^+$	Outcome only (+80 steps)	—	0.651	+3.8pp
PRM-Clip	Clipped average	0.4	0.641	+2.8pp
PRM-Soft†	Continuous penalty	0.4	0.606	−0.7pp
PRM-Mix	Avg+veto+accum	0.5	0.656	+4.3pp
PRM-Mix	Avg+veto+accum	0.7	0.656	+4.3pp
PRM-Exp	Expectile (τ=0.3)	2.0	0.643	+3.0pp
PRM-Exp	**Expectile (τ=0.3)**	**0.5**	**0.676**	**+6.3pp**
PRM-Avg	Average	0.4	0.670	+5.7pp

4 Experimental Setup

Task and data. We train and evaluate on the default-project Countdown pipeline. The warm-start SFT corpus is `Asap7772/cog_behav_all_strategies`; the RLOO prompt set and held-out evaluation set are `asingh15/countdown_tasks_3to4` (instances with 3 or 4 input integers). The text-only SFT checkpoint `asingh15/qwen-sft-countdown-defaultproj` is the starting point for the text-RLOO arm and for the PRM extension; the SFT-TIR checkpoint (`tir_lr_5e-5_epochs_6/epoch_5`) is the starting point for the RLOO-TIR arm.

Model. All runs start from Qwen2.5–0.5B Qwen Team [2025] (or one of its SFT-warm-started descendants). bfloat16 weights throughout, gradient checkpointing on. We use the Qwen tokenizer; no vocabulary changes are needed for the `<calc>`/`<out>`/`<think>`/`<answer>` tags because they are ordinary text the tokenizer encodes as multi-token sequences.

Training hyperparameters. RLOO runs (both text and TIR) use $K = 8$ rollouts per prompt, batch size 128, gradient accumulation 128, learning rate 1×10^{-5} with a constant schedule, weight decay 10^{-4}, KL and entropy coefficients 10^{-3}, 100 update steps. The TIR arm sets gradient clipping to 0.0 because the importance ratio already clips the dominant source of gradient explosion; the text arm uses gradient clipping 1.0. SFT-TIR uses learning rate 5×10^{-5}, 6 epochs, batch size 64, gradient clipping 1.0, weight decay 0.01, warmup ratio 0.05.

Rollout sampling. Training rollouts use temperature 1.0, no top-k/top-p filtering ("constrained decoding" defaults); evaluation uses temperature 0.6, top-p 0.95, top-k 20, max tokens 1,024 per response. For TIR arms the same sampling parameters are passed to the interpreter loop, with `</calc>` and `</answer>` as stop strings.

Evaluation. We score with the project's rule-based verifier in `evaluation/countdown.py`: the verifier returns 0.0 for output without `<answer>` tags, 0.1 for a parseable answer that either uses the wrong number multiset or evaluates to the wrong value, and 1.0 for an answer that uses each provided number exactly once and evaluates exactly to the target. Pass@1 is reported as the *strict* fraction of trials with score 1.0. Each cell of the TIR matrix averages 16 samples per prompt $\times$ 50 prompts = 800 trials.

5 Results

5.1 PRM (Primary Extension)

Table 2 reports the best pass@1 within 80 training steps for each reward formulation. All variants start from the same RLOO-warm-started checkpoint; RLOO-ORM$^+$ is the same policy continued for 80 steps without PRM, serving as a compute-matched outcome-only baseline. Figure 1 shows the full training trajectories.

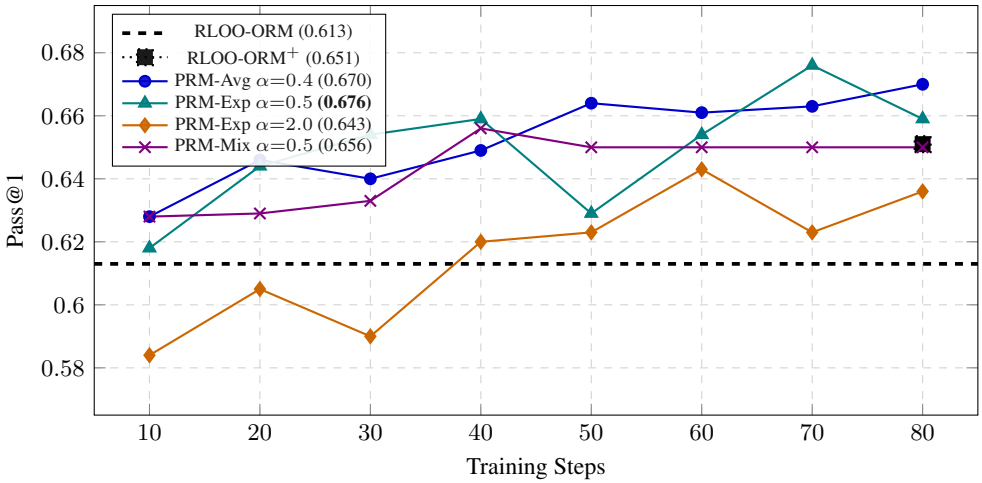

Figure 1: Pass@1 training curves for key PRM variants (all within 80 steps). The dashed horizontal line is the RLOO-ORM outcome-only baseline (0.613). The isolated square marker at step 80 is RLOO-ORM$^+$ (0.651), the same policy continued without PRM for the same compute budget. PRM-Exp (α=0.5) and PRM-Avg converge to ∼0.67–0.68; PRM-Exp at over-scaled α=2.0 stalls below RLOO-ORM$^+$, illustrating the exploration-suppression effect.

Finding 1: PRM consistently outperforms extended outcome-only training. Every calibrated PRM variant (PRM-Avg, PRM-Mix, PRM-Exp with α=0.5) surpasses RLOO-ORM$^+$ (0.651) at matched compute. The best cell, **PRM-Exp at 0.676**, leads extended training by +2.5pp, confirming that step-level process feedback carries genuine signal beyond the final-answer bit and cannot be replicated by simply running RLOO longer.

Finding 2: Reward scale (α) dominates aggregation design. The starkest contrast in Table 2 is within PRM-Exp: scaling α from 0.5 to 2.0 drops pass@1 by 3.3pp with no other change. At α=2.0 the PRM bonus frequently exceeds the ORM outcome signal, eliminating the exploration margin the policy needs to discover valid equation orderings. At α=0.5 the same expectile model recovers fully and slightly outperforms PRM-Avg (0.676 vs. 0.670). Switching aggregation from average to mixture at fixed α=0.5 yields only 0.656 – 1.4pp below PRM-Avg – confirming that *how* step scores are combined matters far less than keeping the bonus bounded relative to the outcome reward.

Finding 3: Heuristic PRM reaches an information bottleneck. PRM-Exp (α=0.5) performs similarly to PRM-Avg (0.676 vs. 0.670) despite encoding a theoretically motivated asymmetry (τ=0.3: error steps weighted 0.7×, correct steps 0.3×). Two substantially different aggregation functions converge to the same pass@1 because both operate on the same three-symbol step-score alphabet $\{+1.0, -1.0, -1.5\}$. Any aggregator on this alphabet is already near-maximally informative; further gains require richer step signals from a learned verifier (Section 7).

5.2 TIR: the 2×2×2 Ablation Matrix (Secondary)

Table 3 gives pass@1 across all eight cells, on the held-out Countdown 3-to-4 split (50 prompts × 16 samples = 800 trials per cell, scored with the strict 1.0 verifier). Three headline observations:

(1) Text-RLOO beats RLOO-TIR head-to-head. With both policies allowed their training-time tool configuration at inference, text-RLOO scores 0.520 vs. RLOO-TIR's 0.413 – tool training costs ∼11 pass@1 points. Both arms use the same dataset, hyperparameters, K, and step budget; the only difference is whether the SFT corpus and rollouts contained `<calc>/<out>` tags.

(2) The best cell is text-RLOO with the tool added at inference only. Row 2 of Table 3 shows that handing the text-RLOO policy a calculator at evaluation time lifts pass@1 from 0.520 to 0.609 – the best cell in the entire matrix. This is interesting because the text-RLOO policy was never trained with

Table 3: pass@1 on the held-out Countdown 3-to-4 split, organized as a $2\times2\times2$ matrix. "Trained tool" = whether the calculator was present during fine-tuning. "Eval tool" = whether the calculator is available at inference. Each cell averages 16 samples per prompt $\times$ 50 prompts = 800 trials. RLOO column dominates SFT column in every row; tool training (rows 3–4) is dominated by text training (rows 1–2) on every metric.

Trained tool	Eval tool	SFT	RLOO	Δ (RLOO$-$SFT)
no	no	0.286	**0.520**	$+0.234$
no	yes	0.325	**0.609**	$+0.284$
yes	no	0.095	0.236	$+0.141$
yes	yes	0.093	0.413	$+0.320$

the calculator in the loop. It suggests that whatever benefit the calculator provides on Countdown is mostly an *inference-time* effect (e.g. catching the occasional arithmetic slip on long expressions); the policy does not need to be adapted to exploit it.

(3) RLOO-TIR is genuinely tool-dependent, not tool-augmented. Disabling the calculator at inference drops RLOO-TIR from 0.413 to 0.236, an 18-point drop. The policy learned to depend on the tool – it did not learn the underlying task. SFT-TIR is the extreme of this: 99.9% of rollouts invoke the calculator and 0% of those invocations fail (we measured zero tool errors across 800 rollouts), but pass@1 is only 0.093. The model mastered the tool protocol while never mastering Countdown's search problem.

Qualitative: where the tokens go. On RLOO-TIR, the average rollout makes 5.05 calculator calls (8 max under the round cap) over $\sim$18–21 tokens per call (the `<calc>EXPR</calc><out>VAL</out>` block plus the cursor reset overhead in vLLM). On SFT-TIR the average is 6.60 calls per rollout, so roughly half the response length is spent on tool exchanges. The model spends its context on the side conversation rather than on `<think>` tokens that explore the search space. The 11-point gap between text-RLOO (no tool, no overhead) and RLOO-TIR (5 tool calls per rollout) is consistent with this – tokens spent in the tool loop are tokens not spent searching.

Qualitative rollout. A text-RLOO policy handed the calculator only at inference (Table 3, row 2):

```
Prompt: nums=[8, 2, 7, 3], target=24.
<think>
8 + 7 = <calc>8+7</calc><out>15</out>, need 9 from {2,3}.
8 - 2 = <calc>8-2</calc><out>6</out>.
7 - 3 = <calc>7-3</calc><out>4</out>.
6 * 4 = <calc>6*4</calc><out>24</out>. Hits target.
</think>
<answer>(8 - 2) * (7 - 3)</answer>
```

This rollout illustrates the inference-time tool benefit: the calculator verifies each candidate combination cheaply, so the model rules out wrong paths without wasting a rollout on a slipped arithmetic step. The same model run without the calculator at inference (row 1) sometimes returns an equally-shaped equation with a slipped intermediate sum.

Why training-time tool exposure hurt. We attribute the negative training-time effect to two mechanisms.

Bottleneck mismatch. Countdown's hard part is the combinatorial search over which operations to apply – for the 3-to-4 split the search tree is small, but the model still has to enumerate it. Arithmetic on small integers is something a 0.5B model already does well inside its head; verifying "$44{+}19{=}63$" with a tool returns the right answer slightly more often, but it does not unlock new search behavior, and it spends tokens that could have been spent searching.

Importance-ratio drift. vLLM and HuggingFace use the same model weights but differ in tokenization edge cases and reduction order in attention; for the same response string they produce slightly

different token-level log-probs. On a text-only RLOO run this difference is small (importance ratios cluster near 1). With <calc>/<out> tags this disagreement is content-dependent and grew large enough that the importance ratio biased to $\sim$0.45 on tool-heavy rollouts, effectively down-weighting exactly the rollouts where the model used the new capability.

6 Discussion

Limitations.

Hyperparameter Search. Due to compute and time constraints, we focused our budget on structurally different reward formulations rather than grid-searching hyperparameters within each variant. Crucial parameters like the reward scale α, the expectile asymmetry τ, and the mixing weights $(w_{\text{avg}}, w_{\min}, w_{\text{sum}})$ were set heuristically or via minimal localized ablations. A comprehensive search might reveal that, e.g., PRM-Mix performs better under a different weighting scheme.

Distribution. Both extensions are tested only on Countdown 3-to-4 and only at the 0.5B scale. Pass@1 on out-of-distribution targets ($T \geq 100$, outside the training-set support) is 0% across all matrix cells, so neither extension extrapolates to targets the warm-start SFT corpus never showed.

Scale. TIR's negative effect is specific to a setting where arithmetic is not the bottleneck. At higher difficulty (e.g. Countdown 5-to-6 with T in the thousands, or GSM8K-style word problems), the same calculator very likely flips to net positive; we did not run that experiment.

Tokenizer mismatch. The $\sim$0.45 importance ratio we observed on RLOO-TIR is a HuggingFace$\leftrightarrow$vLLM artifact, not a fundamental property of the method. Eliminating it would require either generating training rollouts under HuggingFace (slow) or backing out vLLM's exact log-probs at the trainer (engineering work).

PRM scope. The PRM step parser is rule-based; it grades only lines that match `a op b = result`, so multi-operation single-line arithmetic (`((44+19)*2 = 126`) is invisible to it. A learned verifier could grade those too.

Difficulties encountered. The largest single difficulty was diagnosing the importance-ratio drift on RLOO-TIR. The training loop was running, gradients were flowing, the sampler was correctly injecting <out> tags – and pass@1 was trending down. The fix to the gradient was to add the per-sequence importance weight (which the milestone code did not have) and to clip $\log w$ at $\log 2$ on the upside; the unfixable part was the HuggingFace$\leftrightarrow$vLLM mismatch, which we surfaced as a limitation rather than working around. A smaller difficulty was getting the <out> loss mask right at character-offset boundaries: the Qwen tokenizer's fast offset mapping sometimes returns $(0, 0)$ for special tokens with no character provenance, and we initially zeroed those, which produced a near-zero loss at the start of every response.

Broader impacts. Both extensions are modifications to a small math-reasoning pipeline; the broader-impact surface is correspondingly small. The TIR calculator is sandboxed by a regex character whitelist and an `eval` with no builtins, but the general lesson – be careful what you pass to `eval` – obviously generalizes beyond this project.

7 Conclusion

We tested two extensions to the default Countdown RLOO pipeline. PRM (primary) is a structural fix for sparse outcomes that grades each parsed arithmetic step, bounds the partial credit with an asymmetric clip, and adds the result to the outcome reward without changing rollout content; the six-variant ablation identifies **PRM-Exp** (expectile mode, α=0.5, pass@1 $= 0.676$) and **PRM-Avg** (α=0.4, pass@1 $= 0.670$) as the best within the heuristic-scorer family. TIR (secondary) gives the model a calculator backed by a sandboxed Python `eval`, and a $2\times2\times2$ ablation matrix that disentangles training-time from inference-time tool exposure. The matrix reveals a sharper claim than the usual "tools help math": tools help only when they address the task's actual bottleneck. On

Countdown 3-to-4 the bottleneck is search, not arithmetic; training with the calculator in the loop costs 11 pass@1 points, and the best cell is text-RLOO handed a calculator only at inference.

Two concrete future directions follow. (1) *Neural PRM*: the heuristic scorer's three-symbol alphabet $\{+1.0, -1.0, -1.5\}$ is the information bottleneck our ablation identified (Finding 3, Section 5.1). A learned step verifier co-trained with the policy (e.g., a classifier labeled by Monte-Carlo roll-forwards as in Math-Shepherd Wang et al. [2024]) would supply richer per-step gradients. (2) *Scale-up of the TIR matrix*: re-run the eight cells at a problem difficulty where arithmetic is the binding constraint (Countdown 5-to-6 with $T \geq 100$, or GSM8K-style), to test the "tools help only when they address the bottleneck" claim in the opposite regime.

8 Team Contributions

- **Du Li (PRM, primary extension).** Designed and implemented the PRM step parser, number-pool tracing, and the per-step weight table. Ran the full six-variant ablation (PRM-Avg, PRM-Clip, PRM-Soft, PRM-Mix, PRM-Exp at two α scales) that identified average mode as the production formulation and reward scale as the dominant design axis. Generated and analyzed all pass@1 training curves and the ablation table reported in Section 5.1.
- **Michael Yang (TIR, secondary extension; RLOO core).** Implemented the RLOO update worker (importance ratio, analytical KL, entropy bonus, leave-one-out advantages) used by both extensions. Designed and implemented the TIR stack: sandboxed `calc` evaluator, interpreter-in-the-loop sampler, character-offset `<out>` mask, and the `cog_behav` corpus rewriter. Ran the $2\times2\times2$ matrix and produced the analysis in Section 5.2. Also created the poster and presented our work in the poster session.
- **Joint.** SFT warm-start, evaluation harness, and the rule-based Countdown verifier wiring (both extensions hook into the same scorer). Paper writing was split section-by-section.

Changes from proposal. The proposal's headline TIR comparison was a single text-vs-tool delta. We expanded to the $2\times2\times2$ matrix during the milestone after the single comparison surfaced the counterintuitive "tool hurts" result; the additional ablations were what let us attribute the loss to bottleneck mismatch rather than to the implementation. The PRM scope grew substantially: the proposal planned a single average-mode vs. outcome-only comparison, but the development process surfaced five additional formulations (PRM-Clip, PRM-Soft, PRM-Mix, PRM-Exp at two α scales) that were necessary to determine *why* step-level grading works – specifically, that α dominates aggregation design and that heuristic PRMs face an information bottleneck independent of the aggregation choice.

References

Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting reinforce style optimization for learning from human feedback in llms, 2024.

Mohammad Gheshlaghi Azar, Mark Rowland, Bilal Piot, Daniel Guo, Daniele Calandriello, Michal Valko, and Rémi Munos. A general theoretical paradigm to understand learning from human preferences, 2023.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Pal: Program-aided language models, 2023.

Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Minlie Huang, Nan Duan, and Weizhu Chen. Tora: A tool-integrated reasoning agent for mathematical problem solving, 2024.

Ilya Kostrikov, Ashvin Nair, and Sergey Levine. Offline reinforcement learning with implicit Q-learning. In *International Conference on Learning Representations*, 2022. URL `https://openreview.net/forum?id=68n2s9ZJWF8`.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention, 2023.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step, 2023.

Jiayi Pan, Junjie Zhang, Xingyao Wang, Lifan Yuan, Hao Peng, and Alane Suhr. Tinyzero. `https://github.com/Jiayi-Pan/TinyZero`, 2025.

Qwen Team. Qwen2.5 technical report, 2025.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model, 2024.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools, 2023.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024.

Peiyi Wang, Lei Li, Zhihong Shao, R. X. Xu, Damai Dai, Yifei Li, Deli Chen, Y. Wu, and Zhifang Sui. Math-shepherd: Verify and reinforce llms step-by-step without human annotations, 2024.

A Additional Implementation Details

RLOO update worker. The update is computed in `rloo_trainer/rloo_update_worker.py`. On each step the worker receives flattened tensors of shape [batch $\cdot$ K, seq_len] together with rewards, attention masks, an `is_response_token` mask, and (under the sampler's behavior policy) the per-sequence log-probs from vLLM. It computes per-token log-probs under the trainer policy, sums them under the response mask to get $\log \pi_\theta(y \mid x)$, builds the leave-one-out baseline within each group of K, optionally multiplies by $w = \exp(\min(\log \pi_\theta - \log \pi_{\text{old}}, \log 2))$, adds the analytical KL against the frozen reference and an entropy bonus, and steps AdamW with optional gradient clipping. Gradient accumulation requires each microbatch to contain a whole number of K-groups so the leave-one-out baseline remains well-defined.

TIR sampler stop-string handling. vLLM's stop strings are inclusive (`include_stop_str_in_output=True`), so the returned text ends with `</calc>` or `</answer>`. The sampler classifies the stop reason by the trailing tag (`tool_call`, `answer`, or `limit` for max-tokens / EOS with no anchor). On the very rare `malformed_calc` case (stop string fired but no matching `<calc>...</calc>` prefix), the rollout is terminated to avoid an infinite loop.

Reproducibility. All scripts in `rloo_trainer/`, `sft_trainer/`, `tir/`, and `evaluation/` take their hyperparameters from shell scripts whose defaults match the numbers in Section 4. Pass@1 numbers in Table 3 are computed from the eight JSONL files under `local_results/`; a verification script that recomputes them by counting score-1.0 trials per cell is included.